Contents

Information Security (CEC) — Worked Numericals **2**
How to use . . . 2

M06 — TCP flow-control window **2**
What this numerical is . . . 2
Rules . . . 2
Worked example (lecture numbers) . . . 2
Also know (size identity) . . . 3
Exam lines . . . 3

M07 — Stop-and-wait efficiency $U = 1/(1+2a)$ **3**
What this numerical is . . . 3
Formula . . . 3
How to reason qualitatively . . . 4
Sliding-window size (related formula) . . . 4
Exam lines . . . 4

M09 — Classful addressing + network/broadcast (20 hosts) **4**
What this numerical is . . . 4
Formulas / rules . . . 4
Worked example A — Class C fit for 20 machines . . . 4
Worked example B — AND and broadcast . . . 5
Exam lines . . . 5

M10 — CIDR subnetting (20 hosts and 10 hosts) **5**
What this numerical is . . . 5
Powers of two (last octet) . . . 5
Example 1 — 20 machines → mask `255.255.255.224` . . . 6
Example 2 — 10 machines → `/28` . . . 6
Exam lines . . . 6

M11 — TCP sequence / ACK accounting **7**
What this numerical is . . . 7
Rules . . . 7
Tiny data example (as taught) . . . 7
Exam lines . . . 7

M12 — Binary → dotted decimal + CIDR university blocks **7**
Numerical A — binary to IPv4 . . . 7
Numerical B — host `/24` network bits . . . 8
Numerical C — CIDR allocation pool at `194.24.0.0` . . . 8
Exam lines . . . 8

M13 — UDP header hex dump **8**
What this numerical is . . . 8
Dump . . . 8
Exam lines . . . 9

M14 — IPsec AH payload-length field **9**
What this numerical is 9
Worked value from the lecture 9
Exam lines 9

Quick formula card **9**

Information Security (CEC) — Worked Numericals

SWAYAM / CEC Information Security · companion to notes-3
These calculations were flattened or removed when notes-3 kept only key-term tables and exam lines.
Use this pack when the question asks you to **compute**, not only define.

How to use

Each numerical has: **what it is asking**, **formulas / rules**, **worked steps**, and **exam lines**.
Main exam numericals: **M06, M09, M10, M11, M12, M13** (plus short ones from M07, M14).

M06 — TCP flow-control window

What this numerical is

TCP flow control uses a **receiver window (WIN)** that is **independent of ACKs**. You must track buffer space, sequence numbers, ACK numbers, and WIN as data is sent and consumed.

Rules

- Sequence number = first data byte in the segment (byte-stream numbering).
- ACK number = **next byte expected** (not "last byte received").
- WIN = how many more bytes the receiver can still accept **from that next byte**.
- If WIN = 0, sender normally stops (exceptions: urgent data, or a **1-byte window probe** to avoid deadlock).

Worked example (lecture numbers)

Setup: receiver buffer = **4096 bytes**.

Step	What happens	Numbers
1	Sender sends 2048 bytes starting at seq 0; received OK	seq = 0, data = 2048

Step	What happens	Numbers
2	Receiver ACKs; 2048 bytes still free in buffer	ACK = **2048**, WIN = **2048**
3	Sender sends another 2048 bytes	seq = **2048**; buffer now **full**
4	Receiver ACKs with zero window	ACK = **4096**, WIN = **0** → sender **stops**
5	App reads 2K from buffer	free space returns
6	Receiver re-advertises window	ACK = **4096**, WIN = **2048**
7	Sender may continue	e.g. send 1K with seq = **4096**

Also know (size identity)

Max TCP data in one segment (with 20-byte IP + 20-byte TCP headers, no options):

$$65535 - 20 - 20 = 65495 \text{ data bytes}$$

Exam lines

- ACK points to the **next** byte; WIN is remaining buffer from that point.
- WIN=0 is legal; use a **1-byte probe** if a later window update might be lost.
- Flow control here is **end-to-end TCP**, not a data-link ACK coupling.

M07 — Stop-and-wait efficiency $U = 1/(1 + 2a)$

What this numerical is

Stop-and-wait sends **one frame**, then waits for ACK. Efficiency collapses when propagation delay is large compared with transmission time (e.g. satellite).

Formula

Let

- transmission time of one frame = 1 (normalized),
- $a =$ propagation time / transmission time.

Then utilization (efficiency):

$$U = \frac{1}{1 + 2a}$$

Why $2a$? One prop delay to deliver the frame + one prop delay for the ACK (round trip), while the sender is idle waiting.

How to reason qualitatively

Case	Meaning	Efficiency
$a < 1$	First bit arrives before sender finishes the frame	Better
$a > 1$	Whole frame sent before first bit arrives	Poor (satellite-like)

Sliding-window size (related formula)

If sequence-number field has K bits, maximum sender window (Go-Back-N style as taught) is:

$$2^K - 1$$

Example: 3-bit sequence numbers → window up to $2^3 - 1 = 7$.

Exam lines

- Stop-and-wait $U = 1/(1 + 2a)$; large a ⇒ low efficiency.
- Sliding window raises throughput; max window $2^K - 1$.

M09 — Classful addressing + network/broadcast (20 hosts)

What this numerical is

Given a host IP and mask, find **network address** and **broadcast address**. Also choose a classful fit for a small LAN.

Formulas / rules

1. **Network address** = IP **AND** subnet mask (host bits become 0).
2. **Broadcast address** = network address with **all host bits set to 1**.
3. Usable hosts on a subnet:

$$2^n - 2$$

where n = number of **host** bits (subtract network + broadcast).

Worked example A — Class C fit for 20 machines

Need **20** hosts. Class A/B waste huge space. **Class C** default mask `255.255.255.0` has:

$$2^8 - 2 = 254 \text{ usable hosts}$$

Example network: `192.168.1.0`. Assign hosts `192.168.1.1` … `192.168.1.20`.

Worked example B — AND and broadcast

Host: `192.168.1.2`, mask `255.255.255.0`.

Octet	IP	Mask	AND result
1	192	255	192
2	168	255	168
3	1	255	1
4	2	0	**0**

- Network = **`192.168.1.0`**
- Host bits = last octet → all 1s = 255 → broadcast = **`192.168.1.255`**
- Gateway often taught as last usable: **`192.168.1.254`**

Exam lines

- AND with mask → network; host bits all 1 → broadcast.
- Class C: 254 usable; still wasteful for only 20 hosts → motivates **CIDR** (M10).

M10 — CIDR subnetting (20 hosts and 10 hosts)

What this numerical is

Carve a tighter subnet than Class C using a longer mask. Same base net `192.168.1.0`.

Powers of two (last octet)

$$1, 2, 4, 8, 16, 32, 64, 128$$

Rule taught: in the incomplete octet, bits with weight **1** are **network**; bits with weight **0** are **host**.

Block size for last-octet masks:

$$\text{block size} = 256 - (\text{mask last octet})$$

Broadcast = first address of block + block size − 1.

Example 1 — 20 machines → mask `255.255.255.224`

Need at least 20 host addresses ⇒ **5 host bits** (because $2^5 = 32 \geq 20 + 2$).

Last octet: **3 network bits + 5 host bits**.

Network-bit weights: $128 + 64 + 32 = 224$.

Item	Value
Mask	**`255.255.255.224`**
Prefix	`/27` (24 + 3)
Block size	$256 - 224 =$ **32**
Network	`192.168.1.0`
Broadcast	`192.168.1.31`
Usable	`.1` … `.30` (30 usable; only 20 needed ⇒ `.21`–`.30` waste if unused)

Example 2 — 10 machines → `/28`

Need 10 hosts ⇒ **4 host bits** ($2^4 - 2 = 14$ usable).

Last octet: **4 network + 4 host** → `11110000` = $128 + 64 + 32 + 16 = 240$.

Item	Value
Mask	**`255.255.255.240`**
Prefix	**`192.168.1.0/28`**
Usable hosts	$2^4 - 2 =$ **14**
Block size	$256 - 240 =$ **16**
Network	`192.168.1.0`
Broadcast	`192.168.1.15`
First host	`192.168.1.1`
10th assigned	`192.168.1.10`
Unused in block	`.11`–`.14` (4 wasted)

Same meaning: `/28` ⇔ `255.255.255.240`.

Exam lines

- Pick host bits so $2^n - 2 \geq$ required hosts; remaining last-octet bits become network bits.
- Block size $= 256 -$ last mask octet; broadcast = network + block − 1.
- Slash notation and dotted mask are equivalent.

M11 — TCP sequence / ACK accounting

What this numerical is

Track which segments consume sequence-number space.

Rules

Segment type	Consumes a sequence number?
SYN	**Yes** (1)
FIN	**Yes** (1), even if control-only
ACK-only (no data)	**No**
Data of N bytes	**Yes** (N bytes)

Three-way handshake (lecture form):

1. Client → Server: **SYN**, seq = ISN_c
2. Server → Client: **SYN+ACK**, seq = ISN_s, ack = ISN_c+1
3. Client → Server: **ACK**, seq = ISN_c+1, ack = ISN_s+1

Teardown: FIN from each side; final ACK number is **1 + seq of the peer's FIN**.

Tiny data example (as taught)

After connect: client sends **2000 bytes in two segments**; server sends **2000 bytes in one**; client sends one more — the **first three** of those are the main data carriers in the story (then FIN/ACK accounting applies on close).

Exam lines

- SYN and FIN each eat **one** sequence number; pure ACK eats **none**.
- ack = next expected byte = peer's seq + (data length or 1 for SYN/FIN).

M12 — Binary → dotted decimal + CIDR university blocks

Numerical A — binary to IPv4

Binary groups of 8:

`10101100 00010000 11111110 00000001`

Binary	Decimal
10101100	172
00010000	16
11111110	254
00000001	1

Address = **`172.16.254.1`**.

Numerical B — host `/24` network bits

Host `192.168.64.100/24` → 24 network bits, 8 host bits → mask `255.255.255.0` → network `192.168.64.0`.

Numerical C — CIDR allocation pool at `194.24.0.0`

Site	Need	Block (power of two)	Range	Mask
Cambridge	2000	2048	`194.24.0.0`–`194.24.7.255`	`255.255.248.0` (/21)
Edinburgh	1024	1024	`194.24.8.0`–`194.24.11.255`	`255.255.252.0` (/22)
Oxford	4096	4096	**Cannot** start at `194.24.8.0` (not on 4096 boundary); gets `194.24.16.0`–`194.24.31.255`	`255.255.240.0` (/20)

Point of the Oxford case: CIDR blocks must start on an alignment boundary equal to the block size.

Exam lines

- Convert 8-bit groups independently to decimal.
- CIDR gives variable aligned blocks; boundary mistakes matter (Oxford example).

M13 — UDP header hex dump

What this numerical is

Decode an 8-byte UDP header from hex.

Dump

```
CB84 000D 001C 001C
```

UDP header fields (each 16 bits = 4 hex digits):

Field	Hex	Decimal	Meaning
Source port	`CB84`	**52100**	ephemeral client port
Destination port	`000D`	**13**	daytime service
Length	`001C`	**28**	header + data (bytes)
Checksum	`001C`	28	(value as given in dump)

Data length:

$$28 - 8 = 20 \text{ bytes of UDP data}$$

Exam lines

- UDP header is always **8 bytes**; length field includes header+data.
- Hex groups of 4 digits → one 16-bit field.

M14 — IPsec AH payload-length field

What this numerical is

AH **Payload Length** is not "bytes"; it is length of AH in **32-bit words, minus 2**.

Worked value from the lecture

Value **4** means:

$$4 = 3 + 3 - 2$$

Interpretation taught: first **three** 32-bit rows of AH + **three** 32-bit ICV rows, then **minus 2** as required by the field definition → encoded length **4**.

(You do not need to invent other values in the exam unless given; know the **units** and the **−2** rule.)

Exam lines

- AH payload length = (AH size in 32-bit words) − 2.
- Example encoding **4** comes from $3 + 3 - 2$.

Quick formula card

Topic	Formula / rule
TCP usable window story	track seq, ACK=next byte, WIN=free buffer
Stop-and-wait efficiency	$U = 1/(1 + 2a)$
Sliding-window max	$2^K - 1$
Usable hosts	$2^n - 2$
Network address	IP AND mask
Broadcast	host bits all 1
CIDR block size (last octet)	$256-$ mask_last_octet
Max TCP data (no options)	$65535 - 40 = 65495$
UDP data length	length_field − 8
AH payload length	words − 2

Study companion for CEC Information Security. Protocol techniques are for authorized lab/exam use only.